

Assignment 6 – DDL: Database Modeling

Databases for Data Analytics, 2026

Use Case Domain: Premium Car Rentals – a short and long-term vehicle rental service.

	Role	Name
	Customer	Eran Golani
	Designer	Barak Blaish
	Reviewer	Meni Wiengarten

1. Main Use Case

We run a car rental company. Customers can rent cars for a day, a week, or a month. We have different types of cars — economy, SUV, and luxury. Customers pay by credit card based on how long they kept the car. If we find damage when they return it, we charge them for that too.

We need to know at any point which cars are free to rent and which are already out or being repaired. We also need a full record of every rental so we can handle customer questions and disputes. Every customer must be at least 24 years old and have a valid driving license before they can rent.

Customer Requirements

- **R1 – Customer management:** The system should store customer details including name, phone, email, date of birth, and driving license number with its expiry date. Only customers who are 24 or older and have a valid license can rent a car.
- **R2 – Vehicle inventory:** The system should store information about each individual car, including its license plate, category, number of seats, and fuel type. Cars of the same model share the same specs, so those details should be stored once and not repeated for every car.
- **R3 – Vehicle availability:** The system should track whether each car is available, currently rented, under maintenance, or damaged. A car that is not available should not be assignable to a new rental.
- **R4 – Rental orders:** The system should record each rental with the customer, the car, pickup date, planned return date, actual return date, and pickup and return locations.
- **R5 – Pricing and payments:** The system should support different pricing plans such as daily, weekly, and monthly rates. Each rental is charged according to the

chosen plan. If damage was found, the extra charge is added on top. All payment events per rental should be stored.

- **R6 – Damage assessment:** If a car comes back with new damage, the system should record a **description of the damage, the estimated repair cost, and a reference to any photos. It should be** linked to the rental so we know which customer was responsible.
- **R7 – Maintenance tracking:** When a car goes in for service or repair, the system should record what type of maintenance it was, a description, when it started and ended, and who the technician was.

2. Designer's Understanding of the Use Case

Written by: Designer

As the designer, I understand that the main challenge in this system is keeping an accurate and up-to-date picture of every individual car — its condition, availability, and history. Unlike a business that tracks quantities, every car here is unique and must be tracked separately.

I understand that availability is determined by a status field on the car record, not by computing it from rental dates. This covers all the reasons a car might be unavailable, not just active rentals.

I also understand that customers must pass two checks before they can rent: they need to be at least 24 years old and have a valid license. These are business rules enforced at booking time using the `date_of_birth` and `license_expiry_date` stored in the system.

Finally, I understand that pricing is more complex than just a daily or weekly rate. The price also depends on the customer's road experience — how long they have held a license — and their renting score, which starts when they register and changes over time based on their behavior such as damage incidents, fines, and late returns. This means pricing is personal to each customer, not just a fixed plan.

3. Design Discussions

Discussion 1 – Individual Unit Tracking vs. Fleet-Count Tracking

Customer: I want each car tracked separately because every car has its own history and condition.

Designer: I explained that we need two tables — one for individual cars (vehicles) and one for model specifications (vehicle_models). The customer mentioned they want model details stored in one place, so we should not repeat the same specs in every car row. This also makes it easy to update a model's details without changing every car.

Reviewer: I agreed that splitting into two tables is the right normalization decision. I also noted that license_plate should be unique since every car is individually identified.

Final Decision: We agreed on a vehicle_models table for shared specs and a vehicles table for individual cars, connected by a foreign key.

Discussion 2 – Vehicle Availability

Customer: I need to know at any moment whether a car is free to rent.

Designer: I proposed a status field on the vehicles table with values like 'available', 'rented', 'maintenance', and 'damaged'. This is simpler and faster than computing availability from rental date ranges, because a car can become unavailable for reasons other than an active rental.

Reviewer: I approved the status field and pointed out that this is a deliberate choice — status could technically be derived from other tables, but storing it directly avoids slow joins every time someone checks availability. The application must update the field whenever a rental starts or ends.

Final Decision: vehicles.status field with defined values: 'available', 'rented', 'maintenance', 'damaged'.

Discussion 3 – Age and License Validation

Customer: Every customer must be at least 24 years old and have a valid license to rent a car.

Designer: I proposed storing `date_of_birth` and `license_expiry_date` in the `customers` table. The system calculates the customer's age at booking time and blocks the rental if they are under 24 or if their license has expired. I also proposed that `email` and `license_number` should be unique so no two customers can register with the same identity.

Reviewer: I approved and confirmed that age and license checks should be handled by the application, not stored as a column, since age changes daily and a stored value would become outdated.

Final Decision: `customers` table stores `date_of_birth` and `license_expiry_date` with UNIQUE constraints on `email` and `license_number`.

Discussion 4 – Pricing Plans, Renting Score, and Damage Charges

Customer: We have daily, weekly, and monthly pricing. The price also depends on the customer's road experience and their renting score. Damage charges are added on top at the end of the rental.

Designer: I proposed a `pricing_plans` table for the base rate structures. On top of that, I identified two customer-specific factors that affect pricing. First, road experience — derived from the license issue date stored on the customer profile, the system can calculate how many years the customer has been driving. Second, the renting score — a numeric field on the customer record that starts at a default value when they register and is updated automatically after every rental based on incidents like damage reports, fines, and late returns. A customer with a high score and long experience gets a better rate. I also proposed a separate `damage_reports` table linked to the rental, and a `payments` table to record every charge event.

Reviewer: I approved the design and recommended that the renting score update logic be handled by the application after each rental closes, not stored as a manual entry. I also recommended adding a `payment_status` column (pending, completed, failed, refunded) to the `payments` table for dispute handling.

Final Decision: `pricing_plans`, `payments`, and `damage_reports` are three separate tables. The `customers` table stores `renting_score` and `license_issue_date`. Pricing is calculated using the plan rate, road experience, and renting score.

4. Reviewer Summary

Written by: Reviewer

As the reviewer, I verified that the final database design fits the customer requirements and follows normalization principles.

The main improvements included separating the vehicles table from the vehicle_models table to avoid repeated specifications, adding a renting_score and license_issue_date to the customers table to support experience-based pricing, and separating damage_reports from maintenance_records so that customer-linked damage charges and internal service history are tracked independently.

Overall, the final database design provides an organized and reliable solution for managing the car rental company's daily operations.

Key Improvements Identified During Review

- The original design stored model specs directly in the vehicles table. I asked to separate them into a vehicle_models table to avoid repeated data and follow normalization rules.
- The damage_reports table was not linked to a rental at first. I asked to add the rental_id so we can always trace damage back to the responsible customer.
- I recommended adding a renting_score field and license_issue_date to the customers table so that pricing can factor in road experience and past rental behavior.
- I recommended adding a payment_status column to the payments table to support cases where a payment fails and needs to be retried.
- I recommended adding UNIQUE constraints on email and license_number in the customers table to prevent duplicate registrations.
- The vehicles.status field is a deliberate choice — availability could be calculated from other tables, but storing it directly is faster and simpler for daily operations. The application must keep it updated whenever a rental starts or ends.

Overall, the final design covers all customer requirements, is well organized, and is ready to support the company's daily operations.

